AKKA

Scale AI, Efficiently
The Agentic Systems Platform

Akka vs. LangChain

A comparison for teams building agentic AI

October 2026

LangChain is the fastest way to a prototype and the slowest way to production — the moment you ship, you own application availability, recovery planning, and governance. Akka owns all of it, with a contractual SLA.

99.5%	99.9999%	4ms	5B
LANGCHAIN SLA	AKKA SLA	AKKA STATE READS	TOKENS / SEC BENCHMARK

DIMENSION	LANGCHAIN	AKKA
What it is	A developer framework plus LangSmith for observability, evaluation, deployment, and a model gateway; the customer builds the application and owns its availability	The Akka Agentic AI Platform, which guarantees resilience and scalability
Who owns availability	The customer's team	Akka — a contractual 99.9999% SLA, sub-1 min RTO, zero byte RPO
State and memory	Checkpointed to an external database on every step — tens of ms, up to ~150ms as state grows	Embedded in-memory state, active-active replicated: 4ms reads / sub-10ms writes
Governance / EU AI Act	Guardrail middleware coded into each agent can block or pause for approval; the LLM Gateway (beta) enforces spend, rate, and PII policies on model calls; LangSmith adds tracing and online evals; sign-offs are the customer's responsibility	Guardrails, policy enforcement, and hash-chained evidence logging woven into the runtime
Pre-production governance	Not provided	Classify against 241 regulations and frameworks / 1,718 controls, multi-persona sign-offs, Governance Posture Packages
Production runtime	Durable runtime with a limited production track record (LangGraph, stable late 2025); customer-operated, or operated by LangChain on LangSmith Cloud and BYOC	A runtime proven since 2009 across 100,000+ deployments
Infrastructure footprint	More compute for the same transaction volume, plus separately provisioned state, streaming, vector, and observability services	Up to 90% less infrastructure for the same agentic transaction volume — actor concurrency + shared compute
Portability	LangSmith hosting plus a LangChain-specific state layer	Any cloud, on-prem, or sovereign deployment; portable specs; BSL licensing
Who builds & governs it	Software engineers (Python / JS) write agents and their safeguards; LangSmith Fleet builds no-code agents; administrators set model-call policies in the LLM Gateway	Builders and risk/compliance own separate, versioned lifecycles via Akka Specify
Real-time streaming	Bolt-on (Kafka / Flink / Kinesis), separately operated	Built into the runtime — backpressured, petabyte-scale, in-memory, no external broker

Framework vs. Platform: Two Different Problems

Agentic systems run in one of two operating models. **Person-attached** agents run inside a human's session: they inherit the user's identity and stop when the session closes. **Process-attached** agents run unattended, on their own service identity, triggered by events, schedules, or other systems; they must survive crashes, restarts, and infrastructure failures without losing their place. Production agentic AI is overwhelmingly process-attached, and that operating model requires a durable execution substrate. Akka and LangChain's LangGraph both target it, and they meet the requirement differently.

Everything an agentic system needs runs in one Akka runtime, so a system that compiles is ready to run in production. The Akka SDK models the entire agentic system in one place — agents, tools, durable memory, streaming, APIs, and governance — and the platform generates it, tests it, and runs it as one certified service.

What LangGraph provides

LangGraph persists graph state through **checkpointers** that write to an external store — in-memory, SQLite, or Postgres. A graph can resume after an interruption, pause for human review, and carry memory across runs. LangGraph Platform (offered as LangSmith Deployment) adds a task queue, managed persistence, and durable background runs.

What LangGraph does not provide

- **Embedded vs. external state.** LangGraph checkpoints graph state to an external database on every step — each step pays a serialize-and-round-trip cost of tens of ms, up to ~150ms as state grows, compounding across a multi-step agent. Akka holds durable state embedded in memory, active-active replicated, at 4ms reads / sub-10ms writes — durability is a runtime property, not a database round trip.
- **Infrastructure-grade resilience.** No published active-active HA/DR, sub-1-min RTO with zero-byte RPO, or numeric availability SLA. Recovery is a replay from the last checkpoint. LangChain operates that store on LangSmith Cloud and BYOC; on hybrid and self-hosted deployments its HA/DR is the customer's responsibility.
- **A vendor-owned availability guarantee.** Akka owns a contractual 99.9999% SLA on the running system; with LangChain, application availability is owned by the customer.

Stability and operational risk in the durable-execution layer

The checkpointer and durable-execution layer that LangGraph applications depend on has open, maintainer-confirmed defects affecting scaling, availability, and performance — all bug-labeled on GitHub and open as of June 2026:

- **Availability — memory leak in the default mode.** With the default `durability="async"`, checkpointer coroutine chains accumulate and leak memory in a long-running process (#7094).
- **Availability — state loss.** Cancelling a run drops streamed state not yet written as a checkpoint (#5672).
- **Availability — persistence failures.** The Postgres checkpointer throws connection/SSL errors recurring across multiple versions (#3716).
- **Performance & scaling.** Checkpoint serialization produces ~85% storage bloat and ~37.8% token overhead with no opt-out (#7714).

These defects affect scaling, availability, and performance, and a team that adopts the stack owns resolving them in its own deployment. In Akka, durable state is embedded in the runtime, replicated active-active, and operated under a contractual SLA; these failure modes are properties of an external checkpoint store, not the Akka runtime.

Akka Delivers the System; LangChain Delivers Parts

LangChain's portfolio spans the open-source `langchain` framework and LangGraph for durable agent orchestration, the Deep Agents harness, and LangSmith — the commercial agent-engineering platform providing observability, evaluation, LangSmith Deployment for hosting, Fleet for no-code agents, and, in beta, Engine, the LLM Gateway, and Fine-Tuning. The customer assembles these into a running application and owns its availability. Akka ships one pre-integrated platform.

Dimension	LangChain	Akka
Integrated runtime	Components the customer wires together; the application SLA is owned by the customer's team	One pre-integrated runtime; Akka owns a 99.9999% SLA, sub-1 min RTO, and 24/7 SRE with active-active replication
Pricing	Usage billing in LangChain Standard Units: per trace, per vCPU-hour and GiB-hour for managed deployments and their databases, per agent run when self-managed; streaming and governance tooling provisioned separately	Fixed annual fee on shared compute; up to 90% lower infrastructure cost
Governance	Guardrail middleware coded per agent; LLM Gateway (beta) policies on model calls only; LangSmith observability and online evals; conformance built per project	Guardrails, sanitizers, and evidence logging woven into the runtime; conformance proven by Akka Verify

Portability	LangSmith hosting plus a LangChain-specific state layer	Any cloud, on-prem, or sovereign deployment; portable specs; BSL licensing

On LangSmith Cloud and BYOC, LangChain runs the persistence store, scaling, and deployment, and LangSmith supplies observability. The customer still owns application failover and recovery planning, agent-level governance, and compliance tooling, and on hybrid and self-hosted deployments also runs the persistence store and scaling. Akka provides each of these as part of the platform.

Maturity & the Runtime

LangChain's open-source libraries reached v1.0 in October 2025, with a commitment to no breaking changes until 2.0 — a move toward API stability after the frequent restructurings of the 0.x line. The durable runtime production agents depend on has a limited production track record: LangGraph reached its first stable release only in late 2025, and managed hosting (LangSmith Deployment) is newer still, with BYOC generally available on AWS since August 2026 and other clouds planned. LangChain operates Cloud and BYOC deployments; on hybrid and self-hosted deployments the customer operates the agent servers.

Akka's proven runtime

since 2009 across 100,000+ deployments — 52 banks, systems 2 billion people use daily. Built on actor concurrency (200M actors/core), durable sharded in-memory state (4ms reads, 10ms writes, replayable from its event journal), and brokerless backpressured streaming (1-minute RTO, zero-byte RPO). Akka operates this substrate under a contractual SLA.

Availability & DR

LangChain publishes a 99.5% API uptime SLA — for both its Cloud SaaS and its BYOC control plane — measured quarterly with service credits. It does not publish an availability SLA for the customer's application or agents, HA/DR for application state, or automatic failover — application availability is owned by the customer. 99.5% permits ~43.8 hours of downtime per year; Akka's 99.9999% permits ~31 seconds, for the entire running system.

Metric	LangChain	Akka
Published SLA	99.5% API uptime (SaaS & BYOC control plane); LLM Gateway 99.9%	99.9999% whole platform
App / agent availability SLA	Not published	Guaranteed
Who owns availability	The customer	Akka
HA mode	Customer-built	Active-active
RTO / RPO	Customer-implemented	<1 min / zero byte
State during failover	Customer's responsibility	Fully preserved
SLA backing	Service credits (control plane)	Contractual indemnities

Governance and the EU AI Act

The penalties are enforceable now

Violation	Maximum Fine
Prohibited AI practices (Art. 5)	EUR 35M or 7% global turnover
High-risk obligations (Art. 9-15)	EUR 15M or 3% global turnover
Incorrect information to authorities	EUR 7.5M or 1.5% global turnover

High-risk AI carries a 10-year logging-retention obligation under Article 72. Akka indexes 241 regulations and frameworks and 1,718 controls (700 carrying financial penalties) and derives the applicable obligation set automatically.

What LangSmith provides

LangSmith — LangChain's observability and evaluation product — is positioned for EU AI Act readiness: end-to-end **tracing** for action logging (Art. 12), execution-graph **transparency** (Art. 13), **evaluators** for bias, toxicity, hallucination, PII leakage, and prompt injection (Art. 10, 15), **human oversight** via LangGraph's `interrupt` primitive and annotation queues (Art. 14), client-side PII masking of trace data, EU/BYOC/self-hosted residency, and SOC 2 Type II. Developers can code built-in middleware that blocks PII or pauses tool calls for approval, and the LLM Gateway (beta) enforces spend caps, rate limits, and PII redaction on model calls.

Enforcement on agent actions is **coded into each agent**, and the LLM Gateway covers model calls only. LangChain's EU AI Act material is published guidance. Enterprise audit logs record admin and configuration actions for up to 400 days. LangSmith does not provide pre-deployment risk classification, sign-off workflows, or a sealed audit artifact.

EU AI Act, Side by Side

Requirement	LangSmith (LangChain)	Akka
Action logging (Art. 12)	End-to-end tracing of calls, tools, and steps; 14-day or up to 180-day retention	Hash-chained, runtime-witnessed records, retained for the Article 72 horizon
Record integrity	Trace records in LangSmith's store; tamper-evidence not claimed for traces; admin audit logs described as	Tamper-evident, hash-chained

	tamper-resistant	
Transparency (Art. 13)	Execution-graph visualization and step inspection	Self-explanation as a runtime property of every decision
Risk controls and evaluation (Art. 10, 15)	Evaluators for bias, toxicity, hallucination, PII leakage, prompt injection — detect and alert	Inline guardrails, policies, LLMs-as-a-judge, and sanitizers
Enforcement point	Developer-coded middleware blocks or pauses inside each agent; LLM Gateway (beta) enforces administrator policies on model calls only	Enforced inline in the runtime, at the agent boundary
Human oversight (Art. 14)	LangGraph `interrupt` (coded into the graph) plus annotation-queue review	Runtime control plane: pause, override, or redirect any running process
PII handling	Client-side masking of trace data; LLM Gateway redaction on model calls (Enterprise, beta)	Decision, PII scrub, and explanation produced atomically at runtime
Data residency	EU SaaS, BYOC, self-hosted	Any cloud, on-prem, or sovereign deployment
Pre-deployment risk classification	Not provided	241 regulations and frameworks / 1,718 controls, classified before a system ships
Multi-persona sign-off workflows	Not provided	Declarative recipe engine with dossiers and quorum
Sealed audit artifact	Not provided	Governance Posture Package, ready for regulatory handoff

Up to 90% Cheaper to Operate

AI systems built with Akka are up to **90% cheaper to operate than Python-based systems**. The difference is how much infrastructure each approach needs to handle the same volume of agentic transactions: a Python-based stack spreads the work across the application plus separately provisioned services — a database for state, a streaming or message tier, a vector store, and an observability backend, each sized with its own headroom. Akka runs orchestration, agents, memory, streaming, and APIs on one shared-compute runtime and carries the same transaction load on far fewer cores. Three runtime properties produce the gap:

- **Actor-based concurrency** packs far more concurrent work onto each core — ~10 trillion tokens per core per year versus ~2 trillion for comparable solutions, and ~80% less compute than Python-based frameworks. After porting its Python-based systems to Akka, Manulife reported up to 300% more concurrency and 30–50% faster processing.
- **Shared compute** runs APIs, agents, memory, orchestration, and streaming on one runtime rather than separately provisioned services, eliminating the duplicated infrastructure and idle headroom each standalone service carries.
- **Micro-checkpointing** of durable actions resumes work from the last completed step instead of re-running it, minimizing retries — cutting wasted compute and the repeated LLM calls that drive token cost.

The spend is also predictable: a fixed annual fee finance can forecast, rather than consumption metering (per trace, per run, per vCPU-hour) that moves with load.

Continuous AI Intelligence

Akka Optimize keeps improving the AI after it ships. It runs an evaluation loop on your own production traffic, grading live interactions and training smaller specialized models on your proprietary data, so the system keeps getting better as it runs. The loop delivers three results that compound over time:

- **Model leverage** — route each request to the best model from any vendor under your policies, and reserve frontier models for the requests that need them.
- **Data sovereignty** — the specialized models are trained on your proprietary data, owned by you, and run inside your own environment.
- **Cost governance** — full visibility and control of AI spend, with savings that compound as the loop keeps running.

The evaluations run continuously inside Akka and grade traffic from agents wherever they run, in Akka or in third-party harnesses.

Portability

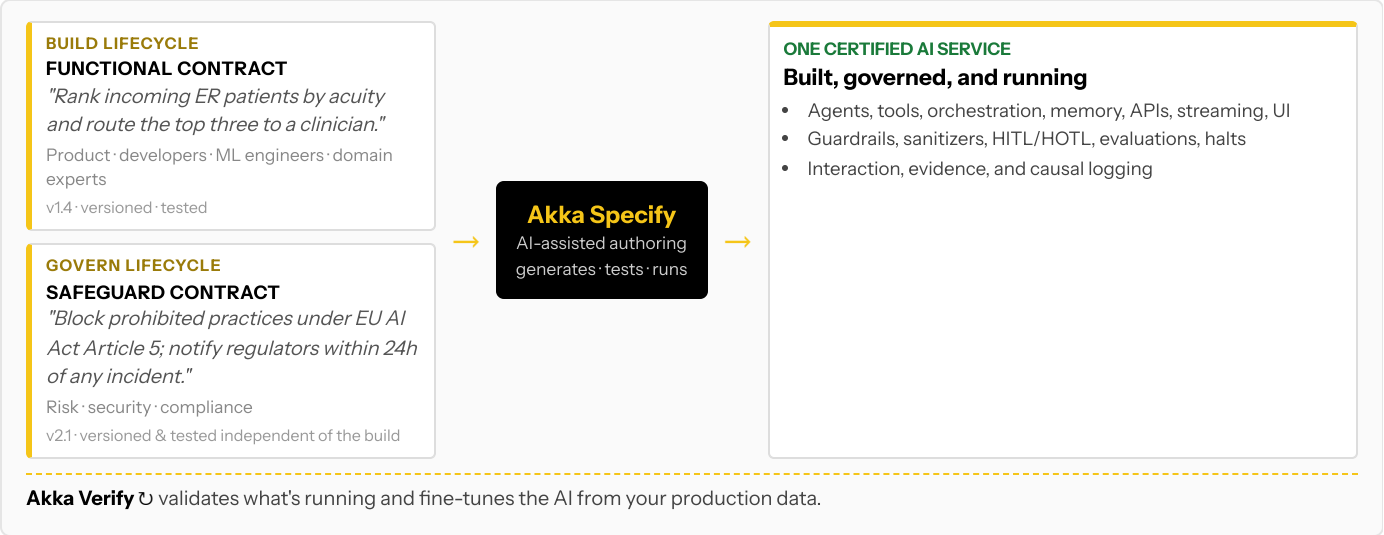
A production LangChain application accumulates dependencies on LangSmith for observability (commercial, per-trace), a state layer built specifically for LangChain's patterns, and separately integrated governance tooling. Migrating off LangChain means replacing each of these. LangSmith BYOC is generally available on AWS only, with other clouds planned; hybrid deployments run the data plane on any Kubernetes cluster.

Akka deploys on:

- Akka's cloud, the customer's hyperscaler VPC, the customer's own Kubernetes, or on-premises
- A sovereign cloud option with country-isolated data, networking, and operations
- Portable specs — the spec is the source of truth and enables agent portability across environments
- BSL licensing

Two Lifecycles, One Certified System

LangChain is a code-first developer framework: building a LangChain application requires software engineers writing Python or TypeScript, and agent safeguards are written by those same engineers inside the application code. LangSmith Fleet lets non-developers build no-code agents, and LLM Gateway administrators set policies on model calls. Risk, security, and compliance have no independent place to own safeguards on agent actions. Akka runs **two independent lifecycles** on one platform, and **Akka Specify** lets each audience work in plain language at its own level of abstraction — a **build lifecycle** (the Dev Spec, authored, versioned, and tested by product, developers, ML engineers, and domain experts) and a **governance lifecycle** (the Eval Matrix, defined, versioned, and tested by risk, security, and compliance *independently of the AI system itself*):



Akka generates, tests, and runs one certified AI service that satisfies both specifications, and **Akka Verify** validates the running system against both. Governance is an independent, versioned, testable lifecycle owned by the people accountable for it — not a developer afterthought in application code — an audience and a workflow LangChain has no equivalent for.

Real-Time Streaming at Petabyte Scale

LangChain has no native stream-processing engine; real-time data pipelines are provisioned and operated separately (Kafka, Flink, Kinesis, or equivalent) and bolted onto the application. Akka's streaming is built into the runtime: continuous, backpressured event flows across components, services, and regions, with no external broker. It processes **petabyte-scale data in memory** with end-to-end backpressure, so fast producers never overwhelm slow consumers and no events are dropped under load. This powers not only agent feedback loops but high-throughput real-time data processing — the engine behind Tubi's real-time hyper-personalization at 5 billion tokens per second. It is a class of real-time, high-volume workload LangChain does not address.

For the Buyer: Risk, Compliance, and Accountability

Buyer concern	LangChain	Akka
Vendor track record	Company founded 2023 (Series B)	Profitable; since 2009 and 100,000+ production deployments (52 banks); Dell Technologies Capital is largest shareholder, a customer, and an AI partner
Certifications & audits	SOC 2 Type II (LangSmith)	19 standards — SOC 2 Type II + public SOC 3, ISO/IEC 27001 & 42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF — plus annual penetration tests, SBOMs, and 40+ documented security policies (trust.akka.io)
Risk transfer	Availability, recovery, and EU AI Act liability sit with the customer	Availability and data-integrity guarantees backed by contractual indemnities
Accountability	You are the integrator and the on-call	Akka owns the SLA and runs 24/7 SRE — one accountable vendor

Budget predictability	Consumption-metered (per trace, per run, per vCPU-hour)	Fixed annual fee finance can forecast
Adoption	—	Incremental; Akka can run alongside existing LangChain/LangGraph work

For an enterprise buyer, the security questionnaire, the liability model, and vendor longevity gate the deal before any feature comparison — and that is where the gap is widest.

Customers Running Agentic and Real-Time Systems on Akka

Manulife
2,000
developers across 100 projects on one governed platform

Tubi
5B tok/s
real-time hyper-personalization engine

Swiggy
71ms
order-assignment AI, ~50% latency reduction

John Deere
1,000+
tractor sensors turned into real-time insight

Verizon
750%
order-processing capacity gain; 6s → 2.4s response

Common Questions

Does our LangChain experience transfer to Akka?
Yes. Agent patterns, prompt engineering, and LLM integration skills apply directly. Akka adds the production layer LangChain does not provide — durable state, HA/DR, governance, and operational guarantees — and Akka Specify lets a team describe the system in plain language and ship it in hours.

We use LangGraph for orchestration. What does Akka add?
Akka provides the same stateful orchestration plus the substrate underneath it: durable in-memory state replicated across the cluster, active-active HA/DR with zero-byte RPO, a vendor-owned availability SLA, and governance woven into the runtime. LangGraph orchestrates; Akka orchestrates and guarantees the running system.

LangGraph Platform deploys our agents. Isn't that production?
LangSmith Deployment hosts applications. LangChain operates Cloud and BYOC deployments and publishes a 99.5% uptime SLA for the SaaS API and the BYOC control plane; BYOC has been generally available on AWS since August 2026. It does not publish an application-level availability SLA, active-active HA/DR with zero-byte RPO, or pre-deployment classification. Akka owns a 99.9999% SLA on the running system, on any cloud.

LangChain is open source. Does Akka create lock-in?
The `langchain` framework is open source. A production LangChain deployment also depends on LangSmith for observability, a LangChain-specific state layer, and separately integrated governance. Akka runs on any cloud or on-premises, keeps the spec portable across environments, and ships under BSL licensing.

Is a platform like Akka heavier than moving fast on a framework?
Time to a prototype and time to production are different measures. Production requirements — HA/DR, governance, compliance, and observability at scale — are where framework-based projects slow down. Akka's spec-driven development ships systems with HA and governance already in place — described in plain language with Akka Specify, then generated, tested, and run by the platform.

Sources

LangChain & LangSmith pricing: langchain.com/pricing; langchain.com/enterprise-pricing (updated 2026-10-01) — Developer $0 / 5k traces; Plus $39/seat / 10k; usage in LangChain Standard Units at $1: 0.0075 LSU/trace; deployment 0.0675 LSU/vCPU-hr, 0.0090 LSU/GiB-hr, database 0.177 LSU/vCPU-hr; self-managed 0.01 LSU/agent run; 0.05 LSU/Fleet run
LangGraph & LangChain v1.0: langchain.com/blog/langchain-langgraph-1dot0 — first stable release, no breaking changes until 2.0
LangSmith deployment & SLA: docs.langchain.com/langsmith/deployments; docs.langchain.com/langsmith/byoc; langchain.com/support-plans — Cloud on AWS and GCP, hybrid on any Kubernetes cluster, BYOC GA on AWS (2026-08-12) with other clouds planned, self-hosted; 99.5% API uptime for SaaS and the BYOC control plane; LLM Gateway 99.9%
LangGraph persistence / checkpointers: langchain-ai.github.io/langgraph — graph state checkpointed to an external store on each step
LangGraph durable-execution defects (open, June 2026): github.com/langchain-ai/langgraph/issues — #7094 default-mode memory leak, #5672 state loss on cancellation, #3716 Postgres checkpointer failures, #7714 checkpoint serialization bloat
Akka trust center: trust.akka.io — 19 compliance standards; SOC 2 Type II + public SOC 3; ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF; annual pen tests, SBOMs, 40+ policies
Akka performance & efficiency: akka.io/blog/go-slow-to-go-fast — Manulife up to 300% more concurrency, 30–50% faster processing after porting Python; ~10T tokens/core/year vs ~2T, ~80% less compute than Python frameworks
LangSmith Shared Responsibility Model: docs.langchain.com/langsmith/shared-responsibility-model — customer owns application-level HA/DR
LangChain on the EU AI Act: langchain.com/blog/langsmith-langchain-oss-eu-ai-act — tracing/logging, evaluators, LangGraph interrupt, residency
LangSmith trace data masking: docs.langchain.com/langsmith/mask-inputs-outputs — client-side PII masking of trace data
LangChain guardrails & LLM Gateway: docs.langchain.com/oss/python/langchain/guardrails; langchain.com/blog/langsmith-llm-gateway-runtime-controls-for-production-agents — PII and human-in-the-loop middleware in application code; Gateway public beta 2026-07-30: spend caps, rate limits, PII redaction on model calls
LangSmith evaluations, audit logs, retention: docs.langchain.com/langsmith/online-evaluations; docs.langchain.com/langsmith/audit-logs; docs.langchain.com/langsmith/administration-overview — online evaluators on production traces; Enterprise audit logs of admin actions, up to 400 days;

extended trace retention up to 180 days from 2026-09-14
Akka platform: 99.9999% availability, active-active HA/DR, sub-1 min RTO, zero byte RPO (contractual indemnities); 241 regulations and frameworks / 1,718 controls / 700 with financial penalties; 100,000+ deployments since 2009